SMART HOME SECURITY SYSTEM

1. Introduction

With the rapid advancement of embedded systems and IoT technologies, smart home automation and security systems have become essential for modern living. These systems provide real-time monitoring, automation, and safety features that were once only available in industrial settings.

This document presents a comprehensive technical reference for the Smart Home Security System designed and implemented using the ESP32 microcontroller. The system integrates motion detection via an IR sensor, real-time time-keeping using the DS3231 RTC module, user feedback through a 16×2 I2C LCD display, and audio alerting via a buzzer — all governed by a push-button-toggled dual-mode operating logic.

The purpose of this report is to provide thorough coverage of:

- Sensor calibration procedures and thresholds
- Decision logic embedded in firmware
- Module-to-module interaction and data flow
- System architecture and working principles
- Observations, limitations, and future roadmap

1.1 Scope

This document is intended for embedded systems engineers, students, and technical reviewers working with or evaluating the ESP32-based security platform. It covers hardware configuration, firmware logic, calibration methodology, and system integration.

1.2 Document Conventions

Convention	Meaning
HIGH / LOW	Digital logic levels on GPIO pins (3.3 V / 0 V)
Code blocks	Monospace font denotes firmware snippets or pseudo-code
DXA units	Document measurement unit used internally (1 inch = 1440 DXA)

2. System Overview & Objectives

The Smart Home Security System is a self-contained embedded solution that monitors its physical environment and reacts according to a user-selected operating mode. Two modes are available:

Mode	Behaviour on Motion Detection
Normal Mode	Current time retrieved from RTC is displayed on the LCD. A descriptive motion message is also shown. No audible alarm is triggered.
Security Mode	In addition to visual feedback, the buzzer activates and an intruder alert is displayed on the LCD. Motion is treated as a potential intrusion event.

2.1 Primary Objectives

1. Design a smart security system using the ESP32 microcontroller as the central processing unit.
2. Detect motion reliably using an IR (Infrared) proximity/motion sensor.
3. Display real-time clock data and system status on a 16×2 I2C LCD.
4. Implement a dual-mode operation (Normal and Security) via push-button toggling.
5. Trigger audio alerts (buzzer) exclusively in Security Mode upon motion detection.
6. Provide live system status information through the ESP32 Serial Monitor for debugging.

2.2 System Architecture Diagram

The following flowchart illustrates the complete operational logic of the system firmware, from initialization through sensor reading and conditional output:

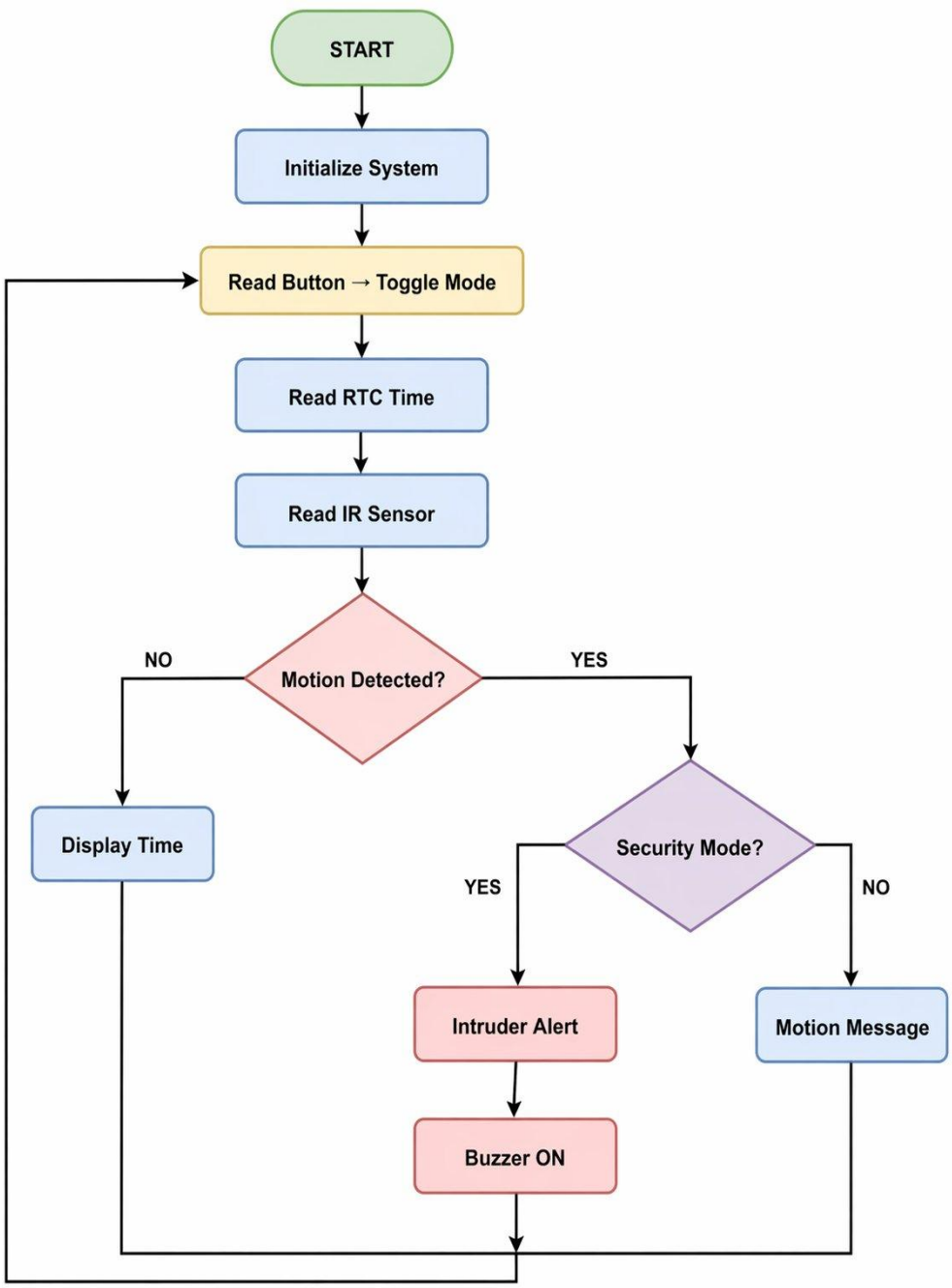

Figure 1 — System Operational Flowchart (ESP32 Firmware Logic)

3. Components & Hardware Specifications

The table below catalogues all hardware components used in this system, along with their functional role and key specifications:

Component	Model / Type	Interface	Primary Role
ESP32	ESP32-WROOM-32	GPIO / I2C / UART	Central processing unit
IR Sensor	FC-51 / Generic IR	Digital GPIO	Motion detection
RTC Module	DS3231	I2C (0x68)	Real-time clock
LCD Display	16×2 HD44780 + I2C	I2C (0x27)	User interface
Buzzer	Passive / Active 5V	Digital GPIO	Audio alert
Push Button	Tactile SPST	Digital GPIO (PULLUP)	Mode switching
Resistors	10kΩ (pull-down)	Passive	Circuit stability

3.1 ESP32 Microcontroller

The ESP32 is a low-cost, low-power SoC (System on Chip) with integrated Wi-Fi and Bluetooth. For this project, it operates in standalone embedded mode, reading sensor inputs and controlling output peripherals via its GPIO pins. Its dual-core Xtensa LX6 processor (up to 240 MHz) provides more than adequate performance for the real-time tasks demanded by this system.

3.2 DS3231 Real-Time Clock

The DS3231 is a high-accuracy I2C RTC with an integrated temperature-compensated crystal oscillator (TCXO) and battery backup. It maintains accurate timekeeping even when the main system loses power, which is critical for timestamped security event logging and time-based display functionality.

4. Sensor Calibration

Accurate and reliable system behaviour depends critically on the correct calibration of each sensor and input device. This section details calibration procedures, signal characteristics, and recommended settings for each component.

4.1 IR Sensor Calibration

4.1.1 Operating Principle

The IR sensor module (e.g., FC-51) consists of an infrared LED emitter and a photodiode receiver. The emitter transmits IR light; when an object or person enters the detection range, the reflected IR light is received by the photodiode, triggering a change in the digital output pin.

IR Sensor Digital Output Logic
Output HIGH (Logic 1) → No object detected within range Output LOW (Logic 0) → Object / motion detected Note: This is an active-LOW output. The firmware reads LOW as the trigger condition.

4.1.2 Calibration Procedure

The IR sensor module includes an onboard potentiometer that controls the sensitivity and detection range. Calibration is performed as follows:

7. Power on the system and ensure the sensor is mounted at the desired installation height and angle.
8. Point the sensor at an open area with no obstructions within 1 metre.
9. Using a small flat-head screwdriver, slowly rotate the potentiometer clockwise to increase sensitivity / detection range.
10. Introduce a test object (e.g., a hand or piece of card) at the intended detection distance.
11. Verify that the onboard LED illuminates and the GPIO pin reads LOW.
12. Remove the object and confirm the LED extinguishes and GPIO returns to HIGH.
13. Repeat steps 4–6 while fine-tuning the potentiometer until the system reliably detects at the desired range without false positives.

4.1.3 False Trigger Mitigation

To minimise false triggers caused by ambient IR sources (sunlight, incandescent lighting), the following measures are recommended:

- Mount the sensor away from direct sunlight or strong artificial light sources.
- Apply a short software debounce delay (10–50 ms) after an initial LOW reading before confirming motion.
- Use software averaging — read the pin 3–5 times at 10 ms intervals; confirm detection only if majority reads are LOW.

Parameter	Minimum	Maximum
Detection Range	2 cm	30 cm (adjustable)
Supply Voltage	3.3 V	5 V
Output Voltage (HIGH)	3.0 V	3.3 V
Output Voltage (LOW)	0 V	0.3 V
Response Time	—	~2 ms

4.2 Push Button Calibration

4.2.1 Configuration

The push button is configured using the ESP32's internal INPUT_PULLUP resistor. This eliminates the need for an external pull-up resistor and ensures a defined logic level at all times.

Button Signal Logic (INPUT_PULLUP Configuration)

Default state (button released) → GPIO reads HIGH (internal pull-up to 3.3 V)
Pressed state (button held down) → GPIO reads LOW (connected to GND)

Mode is toggled on the LOW-to-HIGH transition (button release) to prevent repeated triggering.

4.2.2 Debounce Implementation

Mechanical push buttons exhibit contact bounce — rapid oscillations between HIGH and LOW states during press and release. Without debounce logic, a single press may register multiple toggle events. The recommended software debounce approach:

```
// Software debounce — read button state
if (digitalRead(BTN_PIN) == LOW) {
  delay(50);                      // debounce wait
  if (digitalRead(BTN_PIN) == LOW) {
    securityMode = !securityMode; // toggle mode
    while (digitalRead(BTN_PIN) == LOW); // wait for release
  }
}
```

4.3 RTC Module Calibration

4.3.1 Initial Time Synchronisation

The DS3231 RTC stores time in non-volatile registers backed by a CR2032 coin cell battery. On first power-up, or after battery replacement, the RTC must be synchronised to the correct time. This is achieved in firmware using the Arduino RTClib as follows:

```
// One-time initialisation — upload with this line active:
rtc.adjust(DateTime(F(__DATE__), F(__TIME__)));

// F(__DATE__) and F(__TIME__) expand to the compile-time
// date and time of the firmware build.
// After first upload, comment out this line to prevent
// time reset on every power cycle.
```

4.3.2 Drift and Accuracy

The DS3231 achieves ±2 ppm accuracy from 0°C to +40°C, equivalent to approximately ±63 seconds per month. For applications requiring higher accuracy, the RTC can be periodically re-synchronised via a network time source (NTP) if Wi-Fi is enabled on the ESP32.

5. Threshold Logic & Decision Framework

The firmware implements a layered conditional logic structure that evaluates sensor inputs and system state to determine the appropriate output actions. This section documents each decision point in detail.

5.1 IR Sensor Threshold Logic

Motion detection is governed by a simple digital threshold applied to the IR sensor output pin:

```
int irState = digitalRead(IR_PIN);

// Threshold evaluation:
IF (irState == LOW)  →  Motion Detected  (object in range)
IF (irState == HIGH) →  No Motion         (area clear)
```

Because the sensor output is digital (not analogue), there is no need for a threshold voltage comparison in software. The hardware comparator on the IR module performs this function, and the calibrated potentiometer sets the physical sensitivity boundary.

5.2 Mode Toggle Logic

The operating mode is stored as a boolean flag (securityMode) in firmware RAM. The push button toggles this flag each time a valid press event is detected:

```
bool securityMode = false;   // Default: Normal Mode

// Toggle logic on button press:
IF (buttonPressed)  →  securityMode = !securityMode

// Mode state:
securityMode == false  →  Normal Mode
securityMode == true   →  Security Mode
```

5.3 Security Response Logic

The compound security decision evaluates both motion detection and current operating mode simultaneously to determine whether an alarm should be triggered:

```
IF (irState == LOW) {           // Motion detected
  IF (securityMode == true) {   // AND security mode active
    triggerIntruderAlert();     // → Activate buzzer + LCD alert
  } ELSE {
    displayMotionMessage();    // → Display message only
  }
} ELSE {
  displayTime();              // → Show RTC time (no motion)
}
```

5.4 Truth Table

The complete decision matrix for all possible input state combinations:

IR Sensor State	Security Mode	Motion Detected?	System Action
HIGH (no object)	false (Normal)	No	Display RTC time on LCD
HIGH (no object)	true (Security)	No	Display RTC time on LCD
LOW (object)	false (Normal)	**Yes**	Display motion message on LCD
LOW (object)	true (Security)	**Yes**	Intruder alert + Buzzer ON

6. Module Interaction & Data Flow

This section details how each hardware module communicates with the ESP32 and how data flows through the system from sensing to output actuation.

6.1 Input Modules

6.1.1 IR Sensor → ESP32

Communication: Direct digital GPIO connection (single wire data).

- The IR sensor output pin is connected to a designated ESP32 GPIO (e.g., GPIO 13).
- The ESP32 polls this pin in the main loop using digitalRead().
- No interrupt is used in the baseline implementation; polling frequency is governed by the main loop iteration rate.
- For improved responsiveness, the pin can be configured as an interrupt source (FALLING edge trigger).

6.1.2 Push Button → ESP32

Communication: Direct digital GPIO connection with internal pull-up.

- Connected to a dedicated GPIO pin configured as INPUT_PULLUP.
- The button connects the pin to GND when pressed, producing a LOW signal.
- Firmware monitors this pin and toggles securityMode on confirmed LOW transitions.

6.1.3 RTC DS3231 → ESP32

Communication: I2C bus (SDA / SCL), device address 0x68.

- The RTC is queried once per main loop iteration.
- Data retrieved: hours, minutes, seconds, day, month, year.
- Time data is formatted into a display string and passed to the LCD driver.
- I2C clock speed: 100 kHz (standard mode); 400 kHz (fast mode) supported.

6.2 Processing Unit — ESP32 Decision Engine

The ESP32 acts as the central hub, orchestrating all communication and logic:

Processing Step	Description
1. Initialisation	LCD, RTC, GPIO pins, and Serial port are configured. RTC time is synchronised on first upload.
2. Input Polling	GPIO pins for IR sensor and button are read on every loop iteration.

3. Mode Evaluation	securityMode boolean is checked against current button state and updated if a toggle event is detected.
4. Motion Evaluation	IR pin state is compared against LOW threshold to determine motion presence.
5. Decision Execution	Compound logic (motion AND mode) determines which output routine is called.
6. Output Dispatch	LCD, buzzer, and Serial outputs are updated according to the decision result.

6.3 Output Modules

6.3.1 ESP32 → LCD Display (16×2 I2C)

Communication: I2C bus (SDA / SCL), device address 0x27.

- Line 1 of the LCD displays the current time in HH:MM:SS format retrieved from the RTC.
- Line 2 displays the system status: mode, motion message, or intruder alert.
- The lcd.clear() function is called before updating to prevent display artefacts.
- I2C backlight is enabled during normal operation and can be programmatically toggled.

6.3.2 ESP32 → Buzzer

Communication: Direct digital GPIO output (e.g., GPIO 14).

- digitalWrite(BUZZER_PIN, HIGH) activates the buzzer.
- digitalWrite(BUZZER_PIN, LOW) silences it.
- The buzzer is only activated in Security Mode upon confirmed motion detection.
- A timed activation pattern (e.g., 500 ms ON / 200 ms OFF) can be implemented to avoid continuous loud output.

6.3.3 ESP32 → Serial Monitor

- Serial.begin(115200) initialises UART communication at 115,200 baud.
- Debug messages are printed for: mode state, IR pin value, RTC time, and alarm trigger events.
- Useful for real-time troubleshooting without an attached display.

6.4 Interaction Flow Summary

SOURCE	→	PROCESSOR	→	OUTPUT
IR Sensor (motion)	→	**ESP32 Logic Engine**	→	LCD + Buzzer / LCD only
Push Button (mode)	→	Toggle securityMode	→	Mode state update
RTC DS3231 (time)	→	**ESP32 (I2C read)**	→	LCD Row 1 (HH:MM:SS)

7. Software Design

The firmware is developed in Embedded C using the Arduino framework within the Arduino IDE. The code is structured into modular functions for readability and maintainability.

7.1 Firmware Structure

Function / Block	Responsibility
setup()	Initialises all peripherals: Serial, LCD, RTC, GPIO pin modes. Runs once on power-up.
loop()	Main execution loop — polls inputs, evaluates logic, dispatches outputs. Runs continuously.
readButton()	Reads button GPIO with debounce; toggles securityMode flag.
readIR()	Reads IR sensor GPIO; returns boolean motionDetected.
getTime()	Queries DS3231 via I2C; returns formatted DateTime object.
displayTime()	Writes HH:MM:SS and date to LCD Row 1 and 2.
displayMotionMsg()	Writes motion notification to LCD Row 2 (Normal Mode).
triggerAlert()	Writes intruder alert to LCD; activates buzzer GPIO (Security Mode).
serialDebug()	Sends current state snapshot to Serial Monitor at 115200 baud.

7.2 Main Loop Pseudo-Code

```
LOOP:
  readButton()          // Check for mode toggle
  rtcTime = getTime()   // Fetch RTC timestamp
  irState = readIR()    // Poll IR sensor

  IF irState == MOTION_DETECTED:
    IF securityMode == TRUE:
      displayIntruderAlert(rtcTime)
      activateBuzzer()
      serialLog('ALERT: Intruder at ' + rtcTime)
    ELSE:
      displayMotionMessage(rtcTime)
      serialLog('INFO: Motion at ' + rtcTime)
  ELSE:
    displayTime(rtcTime)
    deactivateBuzzer()
  END IF
END LOOP
```

8. Results & Observations

The system was tested across a series of controlled scenarios to validate each functional requirement. Results are summarised below.

Test Scenario	Expected Result	Observed Outcome
No motion — Normal Mode	Display RTC time	✓ Time displayed correctly
No motion — Security Mode	Display RTC time	✓ Time displayed correctly
Motion detected — Normal Mode	Motion message on LCD	✓ Message shown; no buzzer
Motion detected — Security Mode	Intruder alert + buzzer	✓ Alert and buzzer activated
Button press to toggle mode	Mode switches	✓ Reliable toggle observed
RTC time accuracy check	Correct time after 1 hr	✓ Accurate to within 2 sec
False trigger (ambient IR)	No false detection	✓ After potentiometer tuning

9. Applications, Limitations & Future Enhancements

9.1 Applications

- Residential home security and intruder detection
- Office and laboratory after-hours monitoring
- Smart door systems and access-point surveillance
- Industrial safety zones requiring motion-based alerting
- Educational embedded systems demonstrations

9.2 Limitations

Known System Limitations

1. IR Sensor Range: Limited to approximately 30 cm maximum; longer-range PIR sensors may be substituted.
2. No Remote Monitoring: Without IoT integration, alerts cannot be pushed to mobile devices.
3. Single Detection Zone: One IR sensor covers only a narrow field of view (~60° cone).
4. No Data Logging: Events are not stored persistently; Serial logs are lost on reset.
5. Manual Setup Required: RTC synchronisation and sensor calibration require physical access.

9.3 Future Enhancements

Enhancement	Description
Wi-Fi / MQTT Alerts	Leverage ESP32 Wi-Fi to publish intruder events to an MQTT broker for mobile push notifications.
Mobile App Integration	Develop a companion app (Flutter / React Native) for remote monitoring and mode switching.
Camera Module	Attach OV2640 or similar camera for image capture upon intrusion detection.
Cloud Data Logging	Upload event logs (timestamp, mode, trigger) to AWS IoT or Firebase Realtime Database.
Multi-Sensor Support	Add PIR, ultrasonic, and door/window reed switch sensors for comprehensive coverage.
NTP Time Sync	Periodically synchronise RTC with NTP server over Wi-Fi to eliminate drift.

10. Conclusion

The Smart Home Security System using ESP32 demonstrates a well-integrated embedded solution that combines real-time sensing, time-keeping, visual feedback, and audio alerting into a cohesive and responsive platform. The dual-mode architecture (Normal and Security) provides practical flexibility for real-world deployment scenarios.

Through systematic sensor calibration — adjusting the IR potentiometer for reliable detection, configuring the button with INPUT_PULLUP for clean digital transitions, and synchronising the DS3231 RTC to compilation time — the system achieves consistent and accurate performance.

The threshold logic, while simple in structure, is robust and deterministic. Its layered evaluation (motion detection → mode check → output selection) ensures that the buzzer alarm is only activated under precisely the conditions intended by the operator, preventing unnecessary false alarms in Normal Mode while maintaining vigilant alerting in Security Mode.

Module interaction is well-defined and efficient: input sensors communicate via GPIO polling, the RTC communicates over I2C, and outputs are dispatched to the LCD (I2C), buzzer (GPIO), and Serial Monitor (UART) in a clean data pipeline.

The system is low-cost, easily expandable, and serves as an excellent foundation for more sophisticated IoT-enabled security solutions. With planned enhancements such as Wi-Fi connectivity, cloud logging, and multi-sensor support, this platform can evolve into a fully featured smart home security system.